

Notifications Postback & Opening Chat Thread — Developer Documentation

Overview

This guide explains how a client application should:

1. Detect when a notification/alert has a `defaultAction` and is unread, then call `sendNotificationPostBack`.
2. After the postback, when the item is marked read and carries a `boardId` / `messageId`, open that thread with `redirectToNotificationChatThread`.

Importing

All APIs are imported from `eva-web-sdk` (as declared in the client app `.d.ts`):

```
import { Notification, ChatInterface } from "eva-web-sdk";
```

Notification object paths to check

Use the item returned from `Notification().subscribe(...)`.

What to check	Path
Default action	<code>item.cd.defaultAction</code>
Read state	<code>item.isRead</code>
Action buttons	<code>item.cd.buttons</code>
Thread id after postback	<code>item.cd.ed.payload.boardId</code>
Message id after postback	<code>item.cd.ed.payload.messageId</code>

When to call `sendNotificationPostBack`

Call this when the user clicks the notification/alert row (or an action button) and a postback should run.

Row click — unread item with `defaultAction`

```
if (item?.cd?.defaultAction && !item?.isRead) {
    await notificationApi.sendNotificationPostBack(item, item.cd.defaultAction);
}
```

Button click

```
await notificationApi.sendNotificationPostBack(item, button); // button from item.cd.buttons
```

Signature

```
sendNotificationPostBack(notificationOrAlert, action);
```

Argument	Value
<code>notificationOrAlert</code>	Full notification or alert object from subscribe
<code>action</code>	<code>item.cd.defaultAction</code> or a button from <code>item.cd.buttons</code>

After postback — when to call `redirectToNotificationChatThread`

After a successful postback, the SDK updates that notification/alert so that:

- `item.isRead` becomes `true`
- `item.cd.ed.payload.boardId` is set
- `item.cd.ed.payload.messageId` is set

On a later click (item already read, or no `defaultAction`), open the thread:

```
await notificationApi.redirectToNotificationChatThread(item);
```

`redirectToNotificationChatThread` reads `boardId` from `item.cd.ed.payload.boardId` and opens that chat thread.

For alerts without a `defaultAction`, use:

```
await notificationApi.redirectToLatestAlert(alert);
```

Click decision (client)

User clicks notification / alert



`item.cd.defaultAction` present AND `item.isRead` === false?

- └ Yes → `sendNotificationPostBack(item, item.cd.defaultAction)`
(SDK marks `isRead=true` and stamps `boardId` / `messageId` on `ed.payload`)
- └ No → `redirectToNotificationChatThread(item)`
(for alerts without `defaultAction`: `redirectToLatestAlert(alert)`)

User clicks a button in `item.cd.buttons`

- └ `sendNotificationPostBack(item, button)`

Angular example

Service

```

// notifications.service.ts
import { Injectable, NgZone, OnDestroy } from '@angular/core';
import { BehaviorSubject } from 'rxjs';
import { Notification, ChatInterface } from 'eva-web-sdk';

@Injectable({ providedIn: 'root' })
export class EvaNotificationsService implements OnDestroy {
  private api = Notification();
  private unsubscribeFn?: () => void;

  readonly notifications$ = new BehaviorSubject<any[]>([]);
  readonly alerts$ = new BehaviorSubject<any[]>([]);
  readonly unreadBell$ = new BehaviorSubject<number>(0);

  constructor(private zone: NgZone) {
    ChatInterface().subscribe(() => {
      // Keep chat UI subscribed so postback / redirect results render
    });

    this.unsubscribeFn = this.api.subscribe(
      (notifications: any[], _hasMore: boolean, unreadCount: { bell?: number }, ale
        this.zone.run(() => {
          this.notifications$.next(notifications || []);
          this.alerts$.next(alerts || []);
          this.unreadBell$.next(unreadCount?.bell ?? 0);
        });
    );
  }

  async load(): Promise<void> {
    await this.api.getNotifications();
  }

  /** Notification row click */
  async onNotificationClick(notification: any): Promise<void> {
    if (notification?.cd?.defaultAction && !notification?.isRead) {
      await this.api.sendNotificationPostBack(notification, notification.cd.default
      return;
    }
    await this.api.redirectToNotificationChatThread(notification);
  }

  /** Alert row click */
  async onAlertClick(alert: any): Promise<void> {
    if (alert?.cd?.defaultAction) {
      await this.api.sendNotificationPostBack(alert, alert.cd.defaultAction);
      return;
    }
    await this.api.redirectToLatestAlert(alert);
  }

```

```

}

/** Button click from item.cd.buttons */
async onClick(item: any, button: any): Promise<void> {
  await this.api.sendNotificationPostBack(item, button);
}

ngOnDestroy(): void {
  this.unsubscribeFn?();
}
}

```

Component

```

// notifications-panel.component.ts
import { Component, OnInit } from '@angular/core';
import { EvaNotificationsService } from '../notifications.service';

@Component({
  selector: 'app-notifications-panel',
  templateUrl: './notifications-panel.component.html',
})
export class NotificationsPanelComponent implements OnInit {
  constructor(public notificationsService: EvaNotificationsService) {}

  ngOnInit(): void {
    void this.notificationsService.load();
  }

  onNotificationClick(notification: any): void {
    void this.notificationsService.onNotificationClick(notification);
  }

  onAlertClick(alert: any): void {
    void this.notificationsService.onAlertClick(alert);
  }

  onClick(event: Event, item: any, button: any): void {
    event.stopPropagation();
    void this.notificationsService.onClick(item, button);
  }
}

```

Template

```

<!-- Alerts -->
<article
  *ngFor="let alert of notificationsService.alerts$ | async"
  (click)="onAlertClick(alert)"
>
  <h3>{{ alert?.message?.title }}</h3>
  <button
    *ngFor="let btn of alert?.cd?.buttons"
    type="button"
    (click)="onButtonClick($event, alert, btn)"
  >
    {{ btn.title }}
  </button>
</article>

<!-- Notifications -->
<article
  *ngFor="let n of notificationsService.notifications$ | async"
  (click)="onNotificationClick(n)"
>
  <h3>{{ n?.message?.title }}</h3>
  <p>{{ n?.isRead ? 'Read' : 'Unread' }}</p>
  <button
    *ngFor="let btn of n?.cd?.buttons"
    type="button"
    (click)="onButtonClick($event, n, btn)"
  >
    {{ btn.title }}
  </button>
</article>

```

Practical notes

1. Check `item.cd.defaultAction` and `item.isRead` before deciding postback vs redirect.
2. After postback, reopen with `redirectToNotificationChatThread(item)` (it uses `item.cd.ed.payload.boardId`).
3. Always **stopPropagation** on button clicks so the row handler does not also fire.
4. Ensure `ChatInterface().subscribe` is active so the opened thread appears in chat.
5. In Angular, wrap subscribe updates in `NgZone.run(...)` so the UI refreshes.